

Table of Contents

1. Finding Power of Factorial Divisor
2. Binomial Coefficients
3. Catalan Numbers
4. The Inclusion-Exclusion Principle
5. Burnside's lemma / Pólya enumeration theorem
6. Stars and bars
7. Generating all K-combinations
8. Placing Bishops on a Chessboard
9. Balanced bracket sequences
10. Counting labeled graphs

Translated

From: e-maxx.ru

Finding Power of Factorial Divisor

You are given two numbers n and k . Find the largest power of k x such that $n!$ is divisible by k^x .

Prime k

Let's first consider the case of prime k . The explicit expression for factorial

$$n! = 1 \cdot 2 \cdot 3 \dots (n-1) \cdot n$$

Note that every k -th element of the product is divisible by k , i.e. adds $+1$ to the answer; the number of such elements is $\lfloor \frac{n}{k} \rfloor$.

Next, every k^2 -th element is divisible by k^2 , i.e. adds another $+1$ to the answer (the first power of k has already been counted in the previous paragraph). The number of such elements is $\lfloor \frac{n}{k^2} \rfloor$.

And so on, for every i each k^i -th element adds another $+1$ to the answer, and there are $\lfloor \frac{n}{k^i} \rfloor$ such elements.

The final answer is

$$\lfloor \frac{n}{k} \rfloor + \lfloor \frac{n}{k^2} \rfloor + \dots + \lfloor \frac{n}{k^i} \rfloor + \dots$$

This result is also known as [Legendre's formula](#). The sum is of course finite, since only approximately the first $\log_k n$ elements are not zeros. Thus, the runtime of this algorithm is $O(\log_k n)$.

Implementation

```
int fact_pow (int n, int k) {
    int res = 0;
    while (n) {
        n /= k;
        res += n;
    }
    return res;
}
```

Composite k

The same idea can't be applied directly. Instead we can factor k , representing it as $k = k_1^{p_1} \cdot \dots \cdot k_m^{p_m}$. For each k_i , we find the number of times it is present in $n!$ using the algorithm described above - let's call this value a_i . The answer for composite k will be

$$\min_{i=1\dots m} \frac{a_i}{p_i}$$

Contributors:

[Oleksandr Kulkov](#) (65.31%) [Mariia Mykhailova](#) (28.57%) [Kunal Singh](#) (4.08%) [jxu](#) (2.04%)

Translated

From: e-maxx.ru

Binomial Coefficients

Binomial coefficients $\binom{n}{k}$ are the number of ways to select a set of k elements from n different elements without taking into account the order of arrangement of these elements (i.e., the number of unordered sets).

Binomial coefficients are also the coefficients in the expansion of $(a + b)^n$ (so-called binomial theorem):

$$(a + b)^n = \binom{n}{0} a^n + \binom{n}{1} a^{n-1}b + \binom{n}{2} a^{n-2}b^2 + \dots + \binom{n}{k} a^{n-k}b^k + \dots + \binom{n}{n} b^n$$

It is believed that this formula, as well as the triangle which allows efficient calculation of the coefficients, was discovered by Blaise Pascal in the 17th century. Nevertheless, it was known to the Chinese mathematician Yang Hui, who lived in the 13th century. Perhaps it was discovered by a Persian scholar Omar Khayyam. Moreover, Indian mathematician Pingala, who lived earlier in the 3rd. BC, got similar results. The merit of the Newton is that he generalized this formula for exponents that are not natural.

Calculation

Analytic formula for the calculation:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

This formula can be easily deduced from the problem of ordered arrangement (number of ways to select k different elements from n different elements). First, let's count the number of ordered selections of k elements. There are n ways to select the first element, $n - 1$ ways to select the second element, $n - 2$ ways to select the third element, and so on. As a result, we get the formula of the number of ordered arrangements: $n(n - 1)(n - 2) \dots (n - k + 1) = \frac{n!}{(n-k)!}$. We can easily move to unordered arrangements, noting that each unordered arrangement corresponds to exactly $k!$ ordered arrangements ($k!$ is the number of possible permutations of k elements). We get the final formula by dividing $\frac{n!}{(n-k)!}$ by $k!$.

Recurrence formula (which is associated with the famous "Pascal's Triangle"):

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

It is easy to deduce this using the analytic formula.

Note that for $n < k$ the value of $\binom{n}{k}$ is assumed to be zero.

Properties

Binomial coefficients have many different properties. Here are the simplest of them:

- Symmetry rule:

$$\binom{n}{k} = \binom{n}{n-k}$$

- Factoring in:

$$\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}$$

- Sum over k:

$$\sum_{k=0}^n \binom{n}{k} = 2^n$$

- Sum over n:

$$\sum_{m=0}^n \binom{m}{k} = \binom{n+1}{k+1}$$

- Sum over n and k:

$$\sum_{k=0}^m \binom{n+k}{k} = \binom{n+m+1}{m}$$

- Sum of the squares:

$$\binom{n}{0}^2 + \binom{n}{1}^2 + \dots + \binom{n}{n}^2 = \binom{2n}{n}$$

- Weighted sum:

$$1\binom{n}{1} + 2\binom{n}{2} + \dots + n\binom{n}{n} = n2^{n-1}$$

- Connection with the [Fibonacci numbers](#):

$$\binom{n}{0} + \binom{n-1}{1} + \dots + \binom{n-k}{k} + \dots + \binom{0}{n} = F_{n+1}$$

Calculation

Straightforward calculation using analytical formula

The first, straightforward formula is very easy to code, but this method is likely to overflow even for relatively small values of n and k (even if the answer completely fit into some datatype, the calculation of the intermediate factorials can lead to overflow). Therefore, this method often can only be used with [long arithmetic](#):

```
int C(int n, int k) {
    int res = 1;
    for (int i = n - k + 1; i <= n; ++i)
        res *= i;
    for (int i = 2; i <= k; ++i)
        res /= i;
    return res;
}
```

Improved implementation

Note that in the above implementation numerator and denominator have the same number of factors (k), each of which is greater than or equal to 1. Therefore, we can replace our fraction with a product k fractions, each of which is real-valued. However, on each step after multiplying current answer by each of the next fractions the answer will still be integer (this follows from the property of factoring in).

C++ implementation:

```
int C(int n, int k) {
    double res = 1;
    for (int i = 1; i <= k; ++i)
        res = res * (n - k + i) / i;
    return (int)(res + 0.01);
}
```

Here we carefully cast the floating point number to an integer, taking into account that due to the accumulated errors, it may be slightly less than the true value (for example, 2.99999 instead of 3).

Pascal's Triangle

By using the recurrence relation we can construct a table of binomial coefficients (Pascal's triangle) and take the result from it. The advantage of this method is that intermediate results never exceed the answer and calculating each new table element requires only one addition. The flaw is slow execution for large n and k if you just need a single value and not the whole table (because in order to calculate $\binom{n}{k}$ you will need to build a table of all $\binom{i}{j}$, $1 \leq i \leq n$, $1 \leq j \leq n$, or at least to $1 \leq j \leq \min(i, 2k)$). The time complexity can be considered to be $O(n^2)$.

C++ implementation:

```
const int maxn = ...;
int C[maxn + 1][maxn + 1];
C[0][0] = 1;
for (int n = 1; n <= maxn; ++n) {
    C[n][0] = C[n][n] = 1;
    for (int k = 1; k < n; ++k)
        C[n][k] = C[n - 1][k - 1] + C[n - 1][k];
}
```

If the entire table of values is not necessary, storing only two last rows of it is sufficient (current n -th row and the previous $n - 1$ -th).

Calculation in $O(1)$

Finally, in some situations it is beneficial to precompute all the factorials in order to produce any necessary binomial coefficient with only two divisions later. This can be advantageous when using [long arithmetic](#), when the memory does not allow precomputation of the whole Pascal's triangle.

Computing binomial coefficients modulo m

Quite often you come across the problem of computing binomial coefficients modulo some m .

Binomial coefficient for small n

The previously discussed approach of Pascal's triangle can be used to calculate all values of $\binom{n}{k} \bmod m$ for reasonably small n , since it requires time complexity $O(n^2)$. This approach can handle any modulo, since only addition operations are used.

Binomial coefficient modulo large prime

The formula for the binomial coefficients is

$$\binom{n}{k} = \frac{n!}{k!(n-k)!},$$

so if we want to compute it modulo some prime $m > n$ we get

$$\binom{n}{k} \equiv n! \cdot (k!)^{-1} \cdot ((n-k)!)^{-1} \pmod{m}.$$

First we precompute all factorials modulo m up to $MAXN!$ in $O(MAXN)$ time.

```
factorial[0] = 1;
for (int i = 1; i <= MAXN; i++) {
    factorial[i] = factorial[i - 1] * i % m;
}
```

And afterwards we can compute the binomial coefficient in $O(\log m)$ time.

```
long long binomial_coefficient(int n, int k) {
    return factorial[n] * inverse(factorial[k] * factorial[n - k] % m) % m;
}
```

We even can compute the binomial coefficient in $O(1)$ time if we precompute the inverses of all factorials in $O(\text{MAXN} \log m)$ using the regular method for computing the inverse, or even in $O(\text{MAXN})$ time using the congruence $(x!)^{-1} \equiv ((x-1)!)^{-1} \cdot x^{-1}$ and the method for [computing all inverses](#) in $O(n)$.

```
long long binomial_coefficient(int n, int k) {
    return factorial[n] * inverse_factorial[k] % m * inverse_factorial[n - k] % m;
}
```

Binomial coefficient modulo prime power

Here we want to compute the binomial coefficient modulo some prime power, i.e. $m = p^b$ for some prime p . If $p > \max(k, n-k)$, then we can use the same method as described in the previous section. But if $p \leq \max(k, n-k)$, then at least one of $k!$ and $(n-k)!$ are not coprime with m , and therefore we cannot compute the inverses - they don't exist. Nevertheless we can compute the binomial coefficient.

The idea is the following: We compute for each $x!$ the biggest exponent c such that p^c divides $x!$, i.e. $p^c \mid x!$. Let $c(x)$ be that number. And let $g(x) := \frac{x!}{p^{c(x)}}$. Then we can write the binomial coefficient as:

$$\binom{n}{k} = \frac{g(n)p^{c(n)}}{g(k)p^{c(k)}g(n-k)p^{c(n-k)}} = \frac{g(n)}{g(k)g(n-k)}p^{c(n)-c(k)-c(n-k)}$$

The interesting thing is, that $g(x)$ is now free from the prime divisor p . Therefore $g(x)$ is coprime to m , and we can compute the modular inverses of $g(k)$ and $g(n-k)$.

After precomputing all values for g and c , which can be done efficiently using dynamic programming in $O(n)$, we can compute the binomial coefficient in $O(\log m)$ time. Or precompute all inverses and all powers of p , and then compute the binomial coefficient in $O(1)$.

Notice, if $c(n) - c(k) - c(n-k) \geq b$, then $p^b \mid p^{c(n)-c(k)-c(n-k)}$, and the binomial coefficient is 0.

Binomial coefficient modulo an arbitrary number

Now we compute the binomial coefficient modulo some arbitrary modulus m .

Let the prime factorization of m be $m = p_1^{e_1} p_2^{e_2} \cdots p_h^{e_h}$. We can compute the binomial coefficient modulo $p_i^{e_i}$ for every i . This gives us h different congruences. Since all moduli $p_i^{e_i}$ are coprime, we can apply the [Chinese Remainder Theorem](#) to compute the binomial coefficient modulo the product of the moduli, which is the desired binomial coefficient modulo m .

Binomial coefficient for large n and small modulo

When n is too large, the $O(n)$ algorithms discussed above become impractical. However, if the modulo m is small there are still ways to calculate $\binom{n}{k} \bmod m$.

When the modulo m is prime, there are 2 options:

- [Lucas's theorem](#) can be applied which breaks the problem of computing $\binom{n}{k} \bmod m$ into $\log_m n$ problems of the form $\binom{x_i}{y_i} \bmod m$ where $x_i, y_i < m$. If each reduced coefficient is calculated using precomputed factorials and inverse factorials, the complexity is $O(m + \log_m n)$.

- The method of computing [factorial modulo P](#) can be used to get the required g and c values and use them as described in the section of [modulo prime power](#). This takes $O(m \log_m n)$.

When m is not prime but square-free, the prime factors of m can be obtained and the coefficient modulo each prime factor can be calculated using either of the above methods, and the overall answer can be obtained by the Chinese Remainder Theorem.

When m is not square-free, a [generalization of Lucas's theorem for prime powers](#) can be applied instead of Lucas's theorem.

Practice Problems

- [Codechef - Number of ways](#)
- [Codeforces - Curious Array](#)
- [LightOj - Necklaces](#)
- [HACKEREARTH: Binomial Coefficient](#)
- [SPOJ - Ada and Teams](#)
- [SPOJ - Greedy Walking](#)
- [Uva 13214 - The Robot's Grid](#)
- [SPOJ - Good Predictions](#)
- [SPOJ - Card Game](#)
- [SPOJ - Topper Rama Rao](#)
- [Uva 13184 - Counting Edges and Graphs](#)
- [Codeforces - Anton and School 2](#)
- [Codeforces - Bacterial Melee](#)
- [Codeforces - Points, Lines and Ready-made Titles](#)
- [SPOJ - The Ultimate Riddle](#)
- [CodeChef - Long Sandwich](#)
- [Codeforces - Placing Jinjas](#)

References

- [Blog fishi.devtail.io](#)
- [Question on Mathematics StackExchange](#)
- [Question on CodeChef Discuss](#)

Contributors:

[Soumik Sarkar](#) (34.31%) [prpr](#) (27.2%) [Jakob Kogler](#) (18.83%) [Mariia Mykhailova](#) (5.86%) [Morass](#) (4.18%) [Oleksandr Kulkov](#) (2.51%) [gampu](#) (2.51%)
[Bhuvnesh Jain](#) (1.67%) [Deji Oyerinde](#) (1.67%) [Sunil Sangwan](#) (0.84%) [SiddharthEEE](#) (0.42%)

Translated From: e-maxx.ru

Catalan Numbers

Catalan numbers is a number sequence, which is found useful in a number of combinatorial problems, often involving recursively-defined objects.

This sequence was named after the Belgian mathematician [Catalan](#), who lived in the 19th century. (In fact it was known before to Euler, who lived a century before Catalan).

The first few Catalan numbers C_n (starting from zero):

1, 1, 2, 5, 14, 42, 132, 429, 1430, ...

Application in some combinatorial problems

The Catalan number C_n is the solution for

- Number of correct bracket sequence consisting of n opening and n closing brackets.
- The number of rooted full binary trees with $n + 1$ leaves (vertices are not numbered). A rooted binary tree is full if every vertex has either two children or no children.
- The number of ways to completely parenthesize $n + 1$ factors.
- The number of triangulations of a convex polygon with $n + 2$ sides (i.e. the number of partitions of polygon into disjoint triangles by using the diagonals).
- The number of ways to connect the $2n$ points on a circle to form n disjoint chords.
- The number of [non-isomorphic](#) full binary trees with n internal nodes (i.e. nodes having at least one son).
- The number of monotonic lattice paths from point $(0, 0)$ to point (n, n) in a square lattice of size $n \times n$, which do not pass above the main diagonal (i.e. connecting $(0, 0)$ to (n, n)).
- Number of permutations of length n that can be [stack sorted](#) (i.e. it can be shown that the rearrangement is stack sorted if and only if there is no such index $i < j < k$, such that $a_k < a_i < a_j$).
- The number of [non-crossing partitions](#) of a set of n elements.
- The number of ways to cover the ladder $1 \dots n$ using n rectangles (The ladder consists of n columns, where i^{th} column has a height i).

Calculations

There are two formulas for the Catalan numbers: **Recursive and Analytical**. Since, we believe that all the mentioned above problems are equivalent (have the same solution), for the proof of the formulas below we will choose the task which it is easiest to do.

Recursive formula

$$C_0 = C_1 = 1$$

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}, n \geq 2$$

The recurrence formula can be easily deduced from the problem of the correct bracket sequence.

The leftmost opening parenthesis l corresponds to certain closing bracket r , which divides the sequence into 2 parts which in turn should be a correct sequence of brackets. Thus formula is also divided into 2 parts. If we denote $k = r - l - 1$, then for fixed r , there will be exactly $C_k C_{n-1-k}$ such bracket sequences. Summing this over all admissible k 's, we get the recurrence relation on C_n .

You can also think it in this manner. By definition, C_n denotes number of correct bracket sequences. Now, the sequence may be divided into 2 parts of length k and $n - k$, each of which should be a correct bracket sequence. Example :

$()(())$ can be divided into $()$ and $(())$, but cannot be divided into $()($ and $))$. Again summing over all admissible k 's, we get the recurrence relation on C_n .

C++ implementation

```
const int MOD = ....
const int MAX = ....
int catalan[MAX];
void init() {
    catalan[0] = catalan[1] = 1;
    for (int i=2; i<=n; i++) {
        catalan[i] = 0;
        for (int j=0; j < i; j++) {
            catalan[i] += (catalan[j] * catalan[i-j-1]) % MOD;
            if (catalan[i] >= MOD) {
                catalan[i] -= MOD;
            }
        }
    }
}
```

Analytical formula

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

(here $\binom{n}{k}$ denotes the usual binomial coefficient, i.e. number of ways to select k objects from set of n objects).

The above formula can be easily concluded from the problem of the monotonic paths in square grid. The total number of monotonic paths in the lattice size of $n \times n$ is given by $\binom{2n}{n}$.

Now we count the number of monotonic paths which cross the main diagonal. Consider such paths crossing the main diagonal and find the first edge in it which is above the diagonal. Reflect the path about the diagonal all the way, going after this edge. The result is always a monotonic path in the grid $(n-1) \times (n+1)$. On the other hand, any monotonic path in the lattice $(n-1) \times (n+1)$ must intersect the diagonal. Hence, we enumerated all monotonic paths crossing the main diagonal in the lattice $n \times n$.

The number of monotonic paths in the lattice $(n-1) \times (n+1)$ are $\binom{2n}{n-1}$. Let us call such paths as "bad" paths. As a result, to obtain the number of monotonic paths which do not cross the main diagonal, we subtract the above "bad" paths, obtaining the formula:

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1} \binom{2n}{n}, n \geq 0$$

Reference

- [Catalan Number by Tom Davis](#)

Practice Problems

- [Codechef - PANSTACK](#)
- [Spoj - Skyline](#)
- [UVA - Safe Salutations](#)
- [Codeforces - How many trees?](#)
- [SPOJ - FUNPROB](#)
- [LOJ - 1170 - Counting Perfect BST](#)

- [UVA - 12887 - The Soldier's Dilemma](#)

Contributors:

[likecs](#) (63.16%) [Jakob Kogler](#) (7.37%) [Oleksandr Kulkov](#) (7.37%) [Bhuvnesh Jain](#) (5.26%) [Gabriel Silva Simões](#) (4.21%) [Madhur Chauhan](#) (4.21%)
[Deji Oyerinde](#) (2.11%) [Morass](#) (2.11%) [Turfa Auliarachman](#) (2.11%) [pulkit-singhal](#) (1.05%) [shaurya gupta](#) (1.05%)

Translated

From: e-maxx.ru

The Inclusion-Exclusion Principle

The inclusion-exclusion principle is an important combinatorial way to compute the size of a set or the probability of complex events. It relates the sizes of individual sets with their union.

Statement

The verbal formula

The inclusion-exclusion principle can be expressed as follows:

To compute the size of a union of multiple sets, it is necessary to sum the sizes of these sets **separately**, and then subtract the sizes of all **pairwise** intersections of the sets, then add back the size of the intersections of **triples** of the sets, subtract the size of **quadruples** of the sets, and so on, up to the intersection of **all** sets.

The formulation in terms of sets

The above definition can be expressed mathematically as follows:

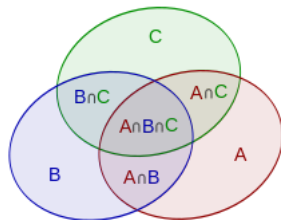
$$\bigcup_{i=1}^n A_i = \sum_{i=1}^n |A_i| - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| + \sum_{1 \leq i < j < k \leq n} |A_i \cap A_j \cap A_k| - \dots + (-1)^{n-1} |A_1 \cap \dots \cap A_n|$$

And in a more compact way:

$$\bigcup_{i=1}^n A_i = \sum_{\emptyset \neq J \subseteq \{1, 2, \dots, n\}} (-1)^{|J|-1} \bigcap_{j \in J} A_j$$

The formulation using Venn diagrams

Let the diagram show three sets A, B and C :



Then the area of their union $A \cup B \cup C$ is equal to the sum of the areas A, B and C less double-covered areas $A \cap B$, $A \cap C$, $B \cap C$, but with the addition of the area covered by three sets $A \cap B \cap C$:

$$S(A \cup B \cup C) = S(A) + S(B) + S(C) - S(A \cap B) - S(A \cap C) - S(B \cap C) + S(A \cap B \cap C)$$

It can also be generalized for an association of n sets.

The formulation in terms of probability theory

If A_i ($i = 1, 2, \dots, n$) are events and $P(A_i)$ the probability of an event from A_i to occur, then the probability of their union (i.e. the probability that at least one of the events occur) is equal to:

$$P\left(\bigcup_{i=1}^n A_i\right) = \sum_{i=1}^n P(A_i) - \sum_{1 \leq i < j \leq n} P(A_i \cap A_j) + \sum_{1 \leq i < j < k \leq n} P(A_i \cap A_j \cap A_k) - \dots + (-1)^{n-1} P(A_1 \cap \dots \cap A_n)$$

And in a more compact way:

$$P\left(\bigcup_{i=1}^n A_i\right) = \sum_{\emptyset \neq J \subseteq \{1, 2, \dots, n\}} (-1)^{|J|-1} P\left(\bigcap_{j \in J} A_j\right)$$

Proof

For the proof it is convenient to use the mathematical formulation in terms of set theory:

$$\bigcup_{i=1}^n A_i = \sum_{\emptyset \neq J \subseteq \{1, 2, \dots, n\}} (-1)^{|J|-1} \bigcap_{j \in J} A_j$$

We want to prove that any element contained in at least one of the sets A_i will occur in the formula only once (note that elements which are not present in any of the sets A_i will never be considered on the right part of the formula).

Consider an element x occurring in $k \geq 1$ sets A_i . We will show it is counted only once in the formula. Note that:

- in terms which $|J| = 1$, the item x will be counted $+k$ times;
- in terms which $|J| = 2$, the item x will be counted $-\binom{k}{2}$ times - because it will be counted in those terms that include two of the k sets containing x ;
- in terms which $|J| = 3$, the item x will be counted $+\binom{k}{3}$ times;
- ...
- in terms which $|J| = k$, the item x will be counted $(-1)^{k-1} \cdot \binom{k}{k}$ times;
- in terms which $|J| > k$, the item x will be counted **zero** times;

This leads us to the following sum of [binomial coefficients](#):

$$T = \binom{k}{1} - \binom{k}{2} + \binom{k}{3} - \dots + (-1)^{i-1} \cdot \binom{k}{i} + \dots + (-1)^{k-1} \cdot \binom{k}{k}$$

This expression is very similar to the binomial expansion of $(1-x)^k$:

$$(1-x)^k = \binom{k}{0} - \binom{k}{1} \cdot x + \binom{k}{2} \cdot x^2 - \binom{k}{3} \cdot x^3 + \dots + (-1)^k \cdot \binom{k}{k} \cdot x^k$$

When $x = 1$, $(1-x)^k$ looks a lot like T . However, the expression has an additional $\binom{k}{0} = 1$, and it is multiplied by -1 . That leads us to $(1-1)^k = 1 - T$. Therefore $T = 1 - (1-1)^k = 1$, what was required to prove. The element is counted only once.

Generalization for calculating number of elements in exactly r sets

Inclusion-exclusion principle can be rewritten to calculate number of elements which are present in zero sets:

$$\bigcap_{i=1}^n \overline{A_i} = \sum_{m=0}^n (-1)^m \sum_{|X|=m} \bigcap_{i \in X} A_i$$

Consider its generalization to calculate number of elements which are present in exactly r sets:

$$\bigcup_{|B|=r} \left[\bigcap_{i \in B} A_i \cap \bigcap_{j \notin B} \overline{A_j} \right] = \sum_{m=r}^n (-1)^{m-r} \binom{m}{r} \sum_{|X|=m} \bigcap_{i \in X} A_i$$

To prove this formula, consider some particular B . Due to basic inclusion-exclusion principle we can say about it that:

$$\bigcap_{i \in B} A_i \cap \bigcap_{j \notin B} \overline{A_j} = \sum_{m=r}^n (-1)^{m-r} \sum_{\substack{|X|=m \\ B \subset X}} \bigcap_{i \in X} A_i$$

The sets on the left side do not intersect for different B , thus we can sum them up directly. Also one should note that any set X will always have coefficient $(-1)^{m-r}$ if it occurs and it will occur for exactly $\binom{m}{r}$ sets B .

Usage when solving problems

The inclusion-exclusion principle is hard to understand without studying its applications.

First, we will look at three simplest tasks "at paper", illustrating applications of the principle, and then consider more practical problems which are difficult to solve without inclusion-exclusion principle.

Tasks asking to "find the **number** of ways" are worth of note, as they sometimes lead to polynomial solutions, not necessarily exponential.

A simple task on permutations

Task: count how many permutations of numbers from 0 to 9 exist such that the first element is greater than 1 and the last one is less than 8.

Let's count the number of "bad" permutations, that is, permutations in which the first element is ≤ 1 and/or the last is ≥ 8 .

We will denote by X the set of permutations in which the first element is ≤ 1 and Y the set of permutations in which the last element is ≥ 8 . Then the number of "bad" permutations, as on the inclusion-exclusion formula, will be:

$$|X \cup Y| = |X| + |Y| - |X \cap Y|$$

After a simple combinatorial calculation, we will get to:

$$2 \cdot 9! + 2 \cdot 9! - 2 \cdot 2 \cdot 8!$$

The only thing left is to subtract this number from the total of $10!$ to get the number of "good" permutations.

A simple task on (0, 1, 2) sequences

Task: count how many sequences of length n exist consisting only of numbers 0, 1, 2 such that each number occurs **at least once**.

Again let us turn to the inverse problem, i.e. we calculate the number of sequences which do **not** contain **at least one** of the numbers.

Let's denote by $A_i (i = 0, 1, 2)$ the set of sequences in which the digit i does **not** occur. The formula of inclusion-exclusion on the number of "bad" sequences will be:

$$|A_0 \cup A_1 \cup A_2| = |A_0| + |A_1| + |A_2| - |A_0 \cap A_1| - |A_0 \cap A_2| - |A_1 \cap A_2| + |A_0 \cap A_1 \cap A_2|$$

- The size of each A_i is 2^n , as each sequence can only contain two of the digits.
- The size of each pairwise intersection $A_i \cap A_j$ is equal to 1, as there will be only one digit to build the sequence.
- The size of the intersection of all three sets is equal to 0, as there will be no digits to build the sequence.

As we solved the inverse problem, we subtract it from the total of 3^n sequences:

$$3^n - (3 \cdot 2^n - 3 \cdot 1 + 0)$$

Number of upper-bound integer sums

Consider the following equation:

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 = 20$$

where $0 \leq x_i \leq 8$ ($i = 1, 2, \dots, 6$).

Task: count the number of solutions to the equation.

Forget the restriction on x_i for a moment and just count the number of nonnegative solutions to this equation. This is easily done using [Stars and Bars](#): we want to break a sequence of 20 units into 6 groups, which is the same as arranging 5 bars and 20 stars:

$$N_0 = \binom{25}{5}$$

We will now calculate the number of "bad" solutions with the inclusion-exclusion principle. The "bad" solutions will be those in which one or more x_i are greater than or equal to 9.

Denote by A_k ($k = 1, 2, \dots, 6$) the set of solutions where $x_k \geq 9$, and all other $x_i \geq 0$ ($i \neq k$) (they may be ≥ 9 or not). To calculate the size of A_k , note that we have essentially the same combinatorial problem that was solved in the two paragraphs above, but now 9 of the units are excluded from the slots and definitely belong to the first group. Thus:

$$|A_k| = \binom{16}{5}$$

Similarly, the size of the intersection between two sets A_k and A_p (for $k \neq p$) is equal to:

$$|A_k \cap A_p| = \binom{7}{5}$$

The size of each intersection of three sets is zero, since 20 units will not be enough for three or more variables greater than or equal to 9.

Combining all this into the formula of inclusions-exceptions and given that we solved the inverse problem, we finally get the answer:

$$\binom{25}{5} - \left(\binom{6}{1} \cdot \binom{16}{5} - \binom{6}{2} \cdot \binom{7}{5} \right)$$

This easily generalizes to d numbers that sum up to s with the restriction $0 \leq x_i \leq b$:

$$\sum_{i=0}^d (-1)^i \binom{d}{i} \binom{s+d-1-(b+1)i}{d-1}$$

As above, we treat binomial coefficients with negative upper index as zero.

Note this problem could also be solved with dynamic programming or generating functions. The inclusion-exclusion answer is computed in $O(d)$ time (assuming math operations like binomial coefficient are constant time), while a simple DP approach would take $O(ds)$ time.

The number of relative primes in a given interval

Task: given two numbers n and r , count the number of integers in the interval $[1; r]$ that are relatively prime to n (their greatest common divisor is 1).

Let's solve the inverse problem - compute the number of not mutually primes with n .

We will denote the prime factors of n as $p_i (i = 1 \dots k)$.

How many numbers in the interval $[1; r]$ are divisible by p_i ? The answer to this question is:

$$\left\lfloor \frac{r}{p_i} \right\rfloor$$

However, if we simply sum these numbers, some numbers will be summarized several times (those that share multiple p_i as their factors). Therefore, it is necessary to use the inclusion-exclusion principle.

We will iterate over all 2^k subsets of p_i s, calculate their product and add or subtract the number of multiples of their product.

Here is a C++ implementation:

```
int solve (int n, int r) {
    vector<int> p;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            p.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        p.push_back (n);

    int sum = 0;
    for (int msk=1; msk<(1<<p.size()); ++msk) {
        int mult = 1,
            bits = 0;
        for (int i=0; i<(int)p.size(); ++i)
            if (msk & (1<<i)) {
                ++bits;
                mult *= p[i];
            }

        int cur = r / mult;
        if (bits % 2 == 1)
            sum += cur;
        else
            sum -= cur;
    }

    return r - sum;
}
```

Asymptotics of the solution is $O(\sqrt{n})$.

The number of integers in a given interval which are multiple of at least one of the given numbers

Given n numbers a_i and number r . You want to count the number of integers in the interval $[1; r]$ that are multiple of at least one of the a_i .

The solution algorithm is almost identical to the one for previous task — construct the formula of inclusion-exclusion on the numbers a_i , i.e. each term in this formula is the number of numbers divisible by a given subset of numbers a_i (in other words, divisible by their **least common multiple**).

So we will now iterate over all 2^n subsets of integers a_i with $O(n \log r)$ operations to find their least common multiple, adding or subtracting the number of multiples of it in the interval. Asymptotics is $O(2^n \cdot n \cdot \log r)$.

The number of strings that satisfy a given pattern

Consider n patterns of strings of the same length, consisting only of letters ($a \dots z$) or question marks. You're also given a number k . A string matches a pattern if it has the same length as the pattern, and at each position, either the corresponding characters are equal or the character in the pattern is a question mark. The task is to count the number of strings that match exactly k of the patterns (first problem) and at least k of the patterns (second problem).

Notice first that we can easily count the number of strings that satisfy at once all of the specified patterns. To do this, simply "cross" patterns: iterate through the positions ("slots") and look at a position over all patterns. If all patterns have a question mark in this position, the character can be any letter from a to z . Otherwise, the character of this position is uniquely defined by the patterns that do not contain a question mark.

Learn now to solve the first version of the problem: when the string must satisfy exactly k of the patterns.

To solve it, iterate and fix a specific subset X from the set of patterns consisting of k patterns. Then we have to count the number of strings that satisfy this set of patterns, and only matches it, that is, they don't match any other pattern. We will use the inclusion-exclusion principle in a slightly different manner: we sum on all supersets Y (subsets from the original set of strings that contain X), and either add to the current answer or subtract it from the number of strings:

$$\text{ans}(X) = \sum_{Y \supseteq X} (-1)^{|Y|-k} \cdot f(Y)$$

Where $f(Y)$ is the number of strings that match Y (at least Y).

(If you have a hard time figuring out this, you can try drawing Venn Diagrams.)

If we sum up on all $\text{ans}(X)$, we will get the final answer:

$$\text{ans} = \sum_{X: |X|=k} \text{ans}(X)$$

However, asymptotics of this solution is $O(3^k \cdot k)$. To improve it, notice that different $\text{ans}(X)$ computations very often share Y sets.

We will reverse the formula of inclusion-exclusion and sum in terms of Y sets. Now it becomes clear that the same set Y would be taken into account in the computation of $\text{ans}(X)$ of $\binom{|Y|}{k}$ sets with the same sign $(-1)^{|Y|-k}$.

$$\text{ans} = \sum_{Y: |Y| \geq k} (-1)^{|Y|-k} \cdot \binom{|Y|}{k} \cdot f(Y)$$

Now our solution has asymptotics $O(2^k \cdot k)$.

We will now solve the second version of the problem: find the number of strings that match **at least** k of the patterns.

Of course, we can just use the solution to the first version of the problem and add the answers for sets with size greater than k . However, you may notice that in this problem, a set $|Y|$ is considered in the formula for all sets with size $\geq k$ which are contained in Y . That said, we can write the part of the expression that is being multiplied by $f(Y)$ as:

$$(-1)^{|Y|-k} \cdot \binom{|Y|}{k} + (-1)^{|Y|-k-1} \cdot \binom{|Y|}{k+1} + (-1)^{|Y|-k-2} \cdot \binom{|Y|}{k+2} + \dots + (-1)^{|Y|-|Y|} \cdot \binom{|Y|}{|Y|}$$

Looking at Graham's (Graham, Knuth, Patashnik. "Concrete mathematics" [1998]), we see a well-known formula for [binomial coefficients](#):

$$\sum_{k=0}^m (-1)^k \cdot \binom{n}{k} = (-1)^m \cdot \binom{n-1}{m}$$

Applying it here, we find that the entire sum of binomial coefficients is minimized:

$$(-1)^{|Y|-k} \cdot \binom{|Y|-1}{|Y|-k}$$

Thus, for this task, we also obtained a solution with the asymptotics $O(2^k \cdot k)$:

$$\text{ans} = \sum_{Y: |Y| \geq k} (-1)^{|Y|-k} \cdot \binom{|Y|-1}{|Y|-k} \cdot f(Y)$$

The number of ways of going from a cell to another

There is a field $n \times m$, and k of its cells are impassable walls. A robot is initially at the cell $(1, 1)$ (bottom left). The robot can only move right or up, and eventually it needs to get into the cell (n, m) , avoiding all obstacles. You need to count the number of ways he can do it.

Assume that the sizes n and m are very large (say, 10^9), and the number k is small (around 100).

For now, sort the obstacles by their coordinate x , and in case of equality — coordinate y .

Also just learn how to solve a problem without obstacles: i.e. learn how to count the number of ways to get from one cell to another. In one axis, we need to go through x cells, and on the other, y cells. From simple combinatorics, we get a formula using [binomial coefficients](#):

$$\binom{x+y}{x}$$

Now to count the number of ways to get from one cell to another, avoiding all obstacles, you can use inclusion-exclusion to solve the inverse problem: count the number of ways to walk through the board stepping at a subset of obstacles (and subtract it from the total number of ways).

When iterating over a subset of the obstacles that we'll step, to count the number of ways to do this simply multiply the number of all paths from starting cell to the first of the selected obstacles, a first obstacle to the second, and so on, and then add or subtract this number from the answer, in accordance with the standard formula of inclusion-exclusion.

However, this will again be non-polynomial in complexity $O(2^k \cdot k)$.

Here goes a polynomial solution:

We will use dynamic programming. For convenience, push $(1,1)$ to the beginning and (n,m) at the end of the obstacles array. Let's compute the numbers $d[i]$ — the number of ways to get from the starting point $(0 - \text{th})$ to $i - \text{th}$, without stepping on any other obstacle (except for i , of course). We will compute this number for all the obstacle cells, and also for the ending one.

Let's forget for a second the obstacles and just count the number of paths from cell 0 to i . We need to consider some "bad" paths, the ones that pass through the obstacles, and subtract them from the total number of ways of going from 0 to i .

When considering an obstacle t between 0 and i ($0 < t < i$), on which we can step, we see that the number of paths from 0 to i that pass through t which have t as the **first obstacle between start and i** . We can compute that as: $d[t]$ multiplied by the number of arbitrary paths from t to i . We can count the number of "bad" ways summing this for all t between 0 and i .

We can compute $d[i]$ in $O(k)$ for $O(k)$ obstacles, so this solution has complexity $O(k^2)$.

The number of coprime quadruples

You're given n numbers: $a_1, a_2, \dots, a_n$. You are required to count the number of ways to choose four numbers so that their combined greatest common divisor is equal to one.

We will solve the inverse problem — compute the number of "bad" quadruples, i.e. quadruples in which all numbers are divisible by a number $d > 1$.

We will use the inclusion-exclusion principle while summing over all possible groups of four numbers divisible by a divisor d .

$$\text{ans} = \sum_{d \geq 2} (-1)^{\text{deg}(d)-1} \cdot f(d)$$

where $\text{deg}(d)$ is the number of primes in the factorization of the number d and $f(d)$ the number of quadruples divisible by d .

To calculate the function $f(d)$, you just have to count the number of multiples of d (as mentioned on a previous task) and use [binomial coefficients](#) to count the number of ways to choose four of them.

Thus, using the formula of inclusions-exclusions we sum the number of groups of four divisible by a prime number, then subtract the number of quadruples which are divisible by the product of two primes, add quadruples divisible by three primes, etc.

The number of harmonic triplets

You are given a number $n \leq 10^6$. You are required to count the number of triples $2 \leq a < b < c \leq n$ that satisfy one of the following conditions:

- or $\gcd(a, b) = \gcd(a, c) = \gcd(b, c) = 1$,
- or $\gcd(a, b) > 1, \gcd(a, c) > 1, \gcd(b, c) > 1$.

First, go straight to the inverse problem — i.e. count the number of non-harmonic triples.

Second, note that any non-harmonic triplet is made of a pair of coprimes and a third number that is not coprime with at least one from the pair.

Thus, the number of non-harmonic triples that contain i is equal the number of integers from 2 to n that are coprimes with i multiplied by the number of integers that are not coprime with i .

Either $\gcd(a, b) = 1 \wedge \gcd(a, c) > 1 \wedge \gcd(b, c) > 1$

or $\gcd(a, b) = 1 \wedge \gcd(a, c) = 1 \wedge \gcd(b, c) > 1$

In both of these cases, it will be counted twice. The first case will be counted when $i = a$ and when $i = b$. The second case will be counted when $i = b$ and when $i = c$. Therefore, to compute the number of non-harmonic triples, we sum this calculation through all i from 2 to n and divide it by 2.

Now all we have left to solve is to learn to count the number of coprimes to i in the interval $[2; n]$. Although this problem has already been mentioned, the above solution is not suitable here — it would require the factorization of each of the integers from 2 to n , and then iterating through all subsets of these primes.

A faster solution is possible with such modification of the sieve of Eratosthenes:

1. First, we find all numbers in the interval $[2; n]$ such that its simple factorization does not include a prime factor twice. We will also need to know, for these numbers, how many factors it includes.
 - To do this we will maintain an array $\text{deg}[i]$ to store the number of primes in the factorization of i , and an array $\text{good}[i]$, to mark either if i contains each factor at most once ($\text{good}[i] = 1$) or not ($\text{good}[i] = 0$). When iterating from 2 to n , if we reach a number that has deg equal to 0, then it is a prime and its deg is 1.
 - During the sieve of Eratosthenes, we will iterate i from 2 to n . When processing a prime number we go through all of its multiples and increase their $\text{deg}[]$. If one of these multiples is multiple of the square of i , then we can put good as false.
2. Second, we need to calculate the answer for all i from 2 to n , i.e., the array $\text{cnt}[]$ — the number of integers not coprime with i .
 - To do this, remember how the formula of inclusion-exclusion works — actually here we implement the same concept, but with inverted logic: we iterate over a component (a product of primes from the factorization) and add or subtract its term on the formula of inclusion-exclusion of each of its multiples.
 - So, let's say we are processing a number i such that $\text{good}[i] = \text{true}$, i.e., it is involved in the formula of inclusion-exclusion. Iterate through all numbers that are multiples of i , and either add or subtract $\lfloor N/i \rfloor$ from their $\text{cnt}[]$ (the signal depends on $\text{deg}[i]$: if $\text{deg}[i]$ is odd, then we must add, otherwise subtract).

Here's a C++ implementation:

```
int n;
bool good[MAXN];
int deg[MAXN], cnt[MAXN];

long long solve() {
    memset(good, 1, sizeof good);
    memset(deg, 0, sizeof deg);
    memset(cnt, 0, sizeof cnt);
```

```

long long ans_bad = 0;
for (int i=2; i<=n; ++i) {
    if (good[i]) {
        if (deg[i] == 0) deg[i] = 1;
        for (int j=1; i*j<=n; ++j) {
            if (j > 1 && deg[i] == 1)
                if (j % i == 0)
                    good[i*j] = false;
            else
                ++deg[i*j];
            cnt[i*j] += (n / i) * (deg[i]%2==1 ? +1 : -1);
        }
    }
    ans_bad += (cnt[i] - 1) * 111 * (n-1 - cnt[i]);
}

return (n-1) * 111 * (n-2) * (n-3) / 6 - ans_bad / 2;
}

```

The asymptotics of our solution is $O(n \log n)$, as for almost every number up to n we make n/i iterations on the nested loop.

The number of permutations without fixed points (derangements)

Prove that the number of permutations of length n without fixed points (i.e. no number i is in position i - also called a derangement) is equal to the following number:

$$n! - \binom{n}{1} \cdot (n-1)! + \binom{n}{2} \cdot (n-2)! - \binom{n}{3} \cdot (n-3)! + \dots \pm \binom{n}{n} \cdot (n-n)!$$

and approximately equal to:

$$\frac{n!}{e}$$

(if you round this expression to the nearest whole number — you get exactly the number of permutations without fixed points)

Denote by A_k the set of permutations of length n with a fixed point at position k ($1 \leq k \leq n$) (i.e. element k is at position k).

We now use the formula of inclusion-exclusion to count the number of permutations with at least one fixed point. For this we need to learn to count sizes of an intersection of sets A_i , as follows:

$$\begin{aligned}
 |A_p| &= (n-1)!, \\
 |A_p \cap A_q| &= (n-2)!, \\
 |A_p \cap A_q \cap A_r| &= (n-3)!, \\
 &\dots,
 \end{aligned}$$

because if we know that the number of fixed points is equal x , then we know the position of x elements of the permutation, and all other $(n-x)$ elements can be placed anywhere.

Substituting this into the formula of inclusion-exclusion, and given that the number of ways to choose a subset of size x from the set of n elements is equal to $\binom{n}{x}$, we obtain a formula for the number of permutations with at least one fixed point:

$$\binom{n}{1} \cdot (n-1)! - \binom{n}{2} \cdot (n-2)! + \binom{n}{3} \cdot (n-3)! - \dots \pm \binom{n}{n} \cdot (n-n)!$$

Then the number of permutations without fixed points is equal to:

$$n! - \binom{n}{1} \cdot (n-1)! + \binom{n}{2} \cdot (n-2)! - \binom{n}{3} \cdot (n-3)! + \dots \pm \binom{n}{n} \cdot (n-n)!$$

Simplifying this expression, we obtain **exact and approximate expressions for the number of permutations without fixed points**:

$$n! \left(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots \pm \frac{1}{n!} \right) \approx \frac{n!}{e}$$

(because the sum in brackets are the first $n + 1$ terms of the expansion in Taylor series e^{-1})

It is worth noting that a similar problem can be solved this way: when you need the fixed points were not among the m first elements of permutations (and not among all, as we just solved). The formula obtained is as the given above accurate formula, but it will go up to the sum of k , instead of n .

Practice Problems

A list of tasks that can be solved using the principle of inclusions-exclusions:

- [UVA #10325 "The Lottery"](#) [difficulty: low]
- [UVA #11806 "Cheerleaders"](#) [difficulty: low]
- [TopCoder SRM 477 "CarelessSecretary"](#) [difficulty: low]
- [TopCoder TCHS 16 "Divisibility"](#) [difficulty: low]
- [SPOJ #6285 NGM2](#), "Another Game With Numbers" [difficulty: low]
- [TopCoder SRM 382 "CharmingTicketsEasy"](#) [difficulty: medium]
- [TopCoder SRM 390 "SetOfPatterns"](#) [difficulty: medium]
- [TopCoder SRM 176 "Deranged"](#) [difficulty: medium]
- [TopCoder SRM 457 "TheHexagonsDivOne"](#) [difficulty: medium]
- [SPOJ #4191 MSKYCODE "Sky Code"](#) [difficulty: medium]
- [SPOJ #4168 SQFREE "Square-free integers"](#) [difficulty: medium]
- [CodeChef "Count Relations"](#) [difficulty: medium]
- [SPOJ - Almost Prime Numbers Again](#)
- [SPOJ - Find number of Pair of Friends](#)
- [SPOJ - Balanced Cow Subsets](#)
- [SPOJ - EASY MATH](#) [difficulty: medium]
- [SPOJ - MOMOS - FEASTOFPIGS](#) [difficulty: easy]
- [Atcoder - Grid 2](#) [difficulty: easy]
- [Codeforces - Count GCD](#)

Contributors:

[Gabriel Silva Simões](#) (81.56%) [Jakob Kogler](#) (5.64%) [Alexander Kulkov](#) (3.04%) [Turf Auliarachman](#) (2.39%) [Anurag Srivastava](#) (1.3%)
[Oleksandr Kulkov](#) (1.3%) [jxu](#) (1.08%) [Bergur Snorrason](#) (0.65%) [Morass](#) (0.65%) [Imiskiew](#) (0.65%) [Abu Hurayra](#) (0.22%) [Bhuvnesh Jain](#) (0.22%)
[L1nkus](#) (0.22%) [Rajat Biswas](#) (0.22%) [Salil Gokhale](#) (0.22%) [Shahnawaz Khan](#) (0.22%) [SiddharthEEE](#) (0.22%) [wiktorkuchta](#) (0.22%)

Translated

From: e-maxx.ru

Burnside's lemma / Pólya enumeration theorem

Burnside's lemma

Burnside's lemma was formulated and proven by **Burnside** in 1897, but historically it was already discovered in 1887 by **Frobenius**, and even earlier in 1845 by **Cauchy**. Therefore it is also sometimes named the **Cauchy-Frobenius lemma**.

Burnside's lemma allows us to count the number of equivalence classes in sets, based on internal symmetry.

Objects and representations

We have to clearly distinguish between the number of objects and the number of representations.

Different representations can correspond to the same objects, but of course any representation corresponds to exactly one object. Consequently the set of all representations is divided into equivalence classes. Our task is to compute the number of objects, or equivalently, the number of equivalence classes. The following example will make the difference between object and representation clearer.

Example: coloring of binary trees

Suppose we have the following problem. We have to count the number of ways to color a rooted binary tree with n vertices with two colors, where at each vertex we do not distinguish between the left and the right children.

Here the set of objects is the set of different colorings of the tree.

We now define the set of representations. A representation of a coloring is a function $f(v)$, which assigns each vertex a color (here we use the colors 0 and 1). The set of representations is the set containing all possible functions of this kind, and its size is obviously equal to 2^n .

At the same time we introduce a partition of this set into equivalence classes.

For example, suppose $n = 3$, and the tree consists of the root 1 and its two children 2 and 3. Then the following functions f_1 and f_2 are considered equivalent.

$$\begin{aligned} f_1(1) &= 0 & f_2(1) &= 0 \\ f_1(2) &= 1 & f_2(2) &= 0 \\ f_1(3) &= 0 & f_2(3) &= 1 \end{aligned}$$

Invariant permutations

Why do these two functions f_1 and f_2 belong to the same equivalence class? Intuitively this is understandable - we can rearrange the children of vertex 1, the vertices 2 and 3, and after such a transformation of the function f_1 it will coincide with f_2 .

But formally this means that there exists an **invariant permutation** π (i.e. a permutation which does not change the object itself, but only its representation), such that:

$$f_2\pi \equiv f_1$$

So starting from the definition of objects, we can find all the invariant permutations, i.e. all permutations which do not change the object when applying the permutation to the representation. Then we can check whether two functions f_1 and f_2 are equivalent (i.e. if they correspond to the same object) by checking the condition $f_2\pi \equiv f_1$ for each invariant permutation (or equivalently $f_1\pi \equiv f_2$). If at least one permutation is found for which the condition is satisfied, then f_1 and f_2 are equivalent, otherwise they are not equivalent.

Finding all such invariant permutations with respect to the object definition is a key step for the application of both Burnside's lemma and the Pólya enumeration theorem. It is clear that these invariant permutations depend on the specific problem, and their finding is a purely heuristic

process based on intuitive considerations. However in most cases it is sufficient to manually find several "basic" permutations, with which all other permutations can be generated (and this part of the work can be shifted to a computer).

It is not difficult to understand that invariant permutations form a **group**, since the product (composition) of invariant permutations is again an invariant permutation. We denote the **group of invariant permutations** by G .

The statement of the lemma

For the formulation of the lemma we need one more definition from algebra. A **fixed point** f for a permutation π is an element that is invariant under this permutation: $f \equiv f\pi$. For example in our example the fixed points are those functions f , which correspond to colorings that do not change when the permutation π is applied to them (i.e. they do not change in the formal sense of the equality of functions). We denote by $I(\pi)$ the **number of fixed points** for the permutation π .

Then **Burnside's lemma** goes as follows: the number of equivalence classes is equal to the sum of the numbers of fixed points with respect to all permutations from the group G , divided by the size of this group:

$$|\text{Classes}| = \frac{1}{|G|} \sum_{\pi \in G} I(\pi)$$

Although Burnside's lemma itself is not so convenient to use in practice (it is unclear how to quickly look for the value $I(\pi)$), it most clearly reveals the mathematical essence on which the idea of calculating equivalence classes is based.

Proof of Burnside's lemma

The proof of Burnside's lemma described here is not important for the practical applications, so it can be skipped on the first reading.

The proof here is the simplest known, and does not use group theory. The proof was published by Kenneth P. Bogart in 1991.

We need to prove the following statement:

$$|\text{Classes}| \cdot |G| = \sum_{\pi \in G} I(\pi)$$

The value on the right side is nothing more than the number of "invariant pairs" (f, π) , i.e. pairs such that $f\pi \equiv f$. It is obvious that we can change the order of summation. We let the sum iterate over all elements f and sum over the values $J(f)$ - the number of permutations for which f is a fixed point.

$$|\text{Classes}| \cdot |G| = \sum_f J(f)$$

To prove this formula we will compose a table with columns labeled with all functions f_i and rows labeled with all permutations π_j . And we fill the cells with $f_i\pi_j$. If we look at the columns in this table as sets, then some of them will coincide, and this means that the corresponding functions f for these columns are also equivalent. Thus the numbers of different (as sets) columns is equal to the number of classes. Incidentally, from the standpoint of group theory, the column labeled with f_i is the orbit of this element. For equivalent elements the orbits coincide, and the number of orbits gives exactly the number of classes.

Thus the columns of the table decompose into equivalence classes. Let us fix a class, and look at the columns in it. First, note that these columns can only contain elements f_i of the equivalence class (otherwise some permutation π_j moved one of the functions into a different equivalence class, which is impossible since we only look at invariant permutations). Secondly each element f_i will occur the same number of times in each column (this also follows from the fact that the columns correspond to equivalent elements). From this we can conclude, that all the columns within the same equivalence class coincide with each other as multisets.

Now fix an arbitrary element f . On the one hand, it occurs in its column exactly $J(f)$ times (by definition). On the other hand, all columns within the same equivalence class are the same as multisets. Therefore within each column of a given equivalence class any element g occurs exactly $J(g)$ times.

Thus if we arbitrarily take one column from each equivalence class, and sum the number of elements in them, we obtain on one hand $|\text{Classes}| \cdot |G|$ (simply by multiplying the number of columns by the number of rows), and on the other hand the sum of the quantities $J(f)$ for

all f (this follows from all the previous arguments):

$$|\text{Classes}| \cdot |G| = \sum_f J(f)$$

Pólya enumeration theorem

The Pólya enumeration theorem is a generalization of Burnside's lemma, and it also provides a more convenient tool for finding the number of equivalence classes. It should be noted that this theorem was already discovered before Pólya by Redfield in 1927, but his publication went unnoticed by mathematicians. Pólya independently came to the same results in 1937, and his publication was more successful.

Here we discuss only a special case of the Pólya enumeration theorem, which will turn out very useful in practice. The general formula of the theorem will not be discussed.

We denote by $C(\pi)$ the number of cycles in the permutation π . Then the following formula (a **special case of the Pólya enumeration theorem**) holds:

$$|\text{Classes}| = \frac{1}{|G|} \sum_{\pi \in G} k^{C(\pi)}$$

k is the number of values that each representation element can take, in the case of the coloring of a binary tree this would be $k = 2$.

Evidence

This formula is a direct consequence of Burnside's lemma. To get it, we just need to find an explicit expression for $I(\pi)$, which appears in the lemma. Recall, that $I(\pi)$ is the number of fixed points in the permutation π .

Thus we consider a permutation π and some element f . During the application of π , the elements in f move via the cycles in the permutation. Since the result should obtain $f \equiv f\pi$, the elements touched by one cycle must all be equal. At the same time different cycles are independent. Thus for each permutation cycle π we can choose one value (among k possible) and thus we get the number of fixed points:

$$I(\pi) = k^{C(\pi)}$$

Application: Coloring necklaces

The problem "Necklace" is one of the classical combinatorial problems. The task is to count the number of different necklaces from n beads, each of which can be painted in one of the k colors. When comparing two necklaces, they can be rotated, but not reversed (i.e. a cyclic shift is permitted).

In this problem we can immediately find the group of invariant permutations:

$$\begin{aligned}\pi_0 &= 123 \dots n \\ \pi_1 &= 23 \dots n1 \\ \pi_2 &= 3 \dots n12 \\ &\dots \\ \pi_{n-1} &= n123 \dots\end{aligned}$$

Let us find an explicit formula for calculating $C(\pi_i)$. First we note, that the permutation π_i has at the j -th position the value $i + j$ (taken modulo n). If we check the cycle structure for π_i . We see that 1 goes to $1 + i$, $1 + i$ goes to $1 + 2i$, which goes to $1 + 3i$, etc., until we come to a number of the form $1 + kn$. Similar statements can be made for the remaining elements. Hence we see that all cycles have the same length, namely $\frac{\text{lcm}(i, n)}{i} = \frac{n}{\text{gcd}(i, n)}$. Thus the number of cycles in π_i will be equal to $\text{gcd}(i, n)$.

Substituting these values into the Pólya enumeration theorem, we obtain the solution:

$$\frac{1}{n} \sum_{i=1}^n k^{\gcd(i,n)}$$

You can leave this formula in this form, or you can simplify it even more. Let transfer the sum so that it iterates over all divisors of n . In the original sum there will be many equivalent terms: if i is not a divisor of n , then such a divisor can be found after computing $\gcd(i, n)$.

Therefore for each divisor $d \mid n$ its term $k^{\gcd(d,n)} = k^d$ will appear in the sum multiple times, i.e. the answer to the problem can be rewritten as

$$\frac{1}{n} \sum_{d \mid n} C_d k^d,$$

where C_d is the number of such numbers i with $\gcd(i, n) = d$. We can find an explicit expression for this value. Any such number i has the form $i = dj$ with $\gcd(j, n/d) = 1$ (otherwise $\gcd(i, n) > d$). So we can count the number of j with this behavior. [Euler's phi function](#) gives us the result $C_d = \phi(n/d)$, and therefore we get the answer:

$$\frac{1}{n} \sum_{d \mid n} \phi\left(\frac{n}{d}\right) k^d$$

Application: Coloring a torus

Quite often we cannot obtain an explicit formula for the number of equivalence classes. In many problems the number of permutations in a group can be too large for manual calculations and it is not possible to compute analytically the number of cycles in them.

In that case we should manually find several "basic" permutations, so that they can generate the entire group G . Next we can write a program that will generate all permutations of the group G , count the number of cycles in them, and compute the answer with the formula.

Consider the example of the problem for coloring a torus. There is a checkered sheet of paper $n \times m$ ($n < m$), some of the cells are black. Then a cylinder is obtained from this sheet by gluing together the two sides with lengths m . Then a torus is obtained from the cylinder by gluing together the two circles (top and bottom) without twisting. The task is to compute the number of different colored tori, assuming that we cannot see the glued lines, and the torus can be turned and turned.

We again start with a piece of $n \times m$ paper. It is easy to see that the following types of transformations preserve the equivalence class: a cyclic shift of the rows, a cyclic shift of the columns, and a rotation of the sheet by 180 degrees. It is also easy to see, that these transformations can generate the entire group of invariant transformations. If we somehow number the cells of the paper, then we can write three permutations p_1, p_2, p_3 corresponding to these types of transformation.

Next it only remains to generate all permutations obtained as a product. It is obvious that all such permutations have the form $p_1^{i_1} p_2^{i_2} p_3^{i_3}$ where $i_1 = 0 \dots m-1, i_2 = 0 \dots n-1, i_3 = 0 \dots 1$.

Thus we can write the implementations to this problem.

```
using Permutation = vector<int>;

void operator*=(Permutation& p, Permutation const& q) {
    Permutation copy = p;
    for (int i = 0; i < p.size(); i++)
        p[i] = copy[q[i]];
}

int count_cycles(Permutation p) {
    int cnt = 0;
    for (int i = 0; i < p.size(); i++) {
        if (p[i] != -1) {
            cnt++;
            for (int j = i; p[j] != -1;) {
                int next = p[j];
                p[j] = -1;
                j = next;
            }
        }
    }
    return cnt;
}
```

```

int solve(int n, int m) {
    Permutation p(n*m), p1(n*m), p2(n*m), p3(n*m);
    for (int i = 0; i < n*m; i++) {
        p[i] = i;
        p1[i] = (i % n + 1) % n + i / n * n;
        p2[i] = (i / n + 1) % m * n + i % n;
        p3[i] = (m - 1 - i / n) * n + (n - 1 - i % n);
    }

    set<Permutation> s;
    for (int i1 = 0; i1 < n; i1++) {
        for (int i2 = 0; i2 < m; i2++) {
            for (int i3 = 0; i3 < 2; i3++) {
                s.insert(p);
                p *= p3;
            }
            p *= p2;
        }
        p *= p1;
    }

    int sum = 0;
    for (Permutation const& p : s) {
        sum += 1 << count_cycles(p);
    }
    return sum / s.size();
}

```

Practice Problems

- [CSES - Counting Necklaces](#)
- [CSES - Counting Grids](#)
- [Codeforces - Buildings](#)
- [CS Academy - Cube Coloring](#)
- [Codeforces - Side Transmutations](#)
- [LightOJ - Necklace](#)
- [POJ - Necklace of Beads](#)
- [CodeChef - Lucy and Flowers](#)
- [HackerRank - Count the Necklaces](#)
- [POJ - Magic Bracelet](#)
- [SPOJ - Sorting Machine](#)
- [Project Euler - Pizza Toppings](#)
- [ICPC 2011 SERCP - Alphabet Soup](#)
- [GCPC 2017 - Buildings](#)

Contributors:

[Jakob Kogler](#) (89.75%)
 [AYUSH KUMAR TIWARI](#) (3.53%)
 [Oleksandr Kulkov](#) (2.12%)
 [Mirco Paul](#) (1.77%)
 [eeegnu](#) (1.06%)
 [wiktorkuchta](#) (1.06%)
[Ashutosh Kumar Verma](#) (0.35%)
 [NhatMinh0208](#) (0.35%)

Original

Stars and bars

Stars and bars is a mathematical technique for solving certain combinatorial problems. It occurs whenever you want to count the number of ways to group identical objects.

Theorem

The number of ways to put n identical objects into k labeled boxes is

$$\binom{n+k-1}{n}.$$

The proof involves turning the objects into stars and separating the boxes using bars (therefore the name). E.g. we can represent with $\star|\star\star||\star\star$ the following situation: in the first box is one object, in the second box are two objects, the third one is empty and in the last box are two objects. This is one way of dividing 5 objects into 4 boxes.

It should be pretty obvious, that every partition can be represented using n stars and $k-1$ bars and every stars and bars permutation using n stars and $k-1$ bars represents one partition. Therefore the number of ways to divide n identical objects into k labeled boxes is the same number as there are permutations of n stars and $k-1$ bars. The [Binomial Coefficient](#) gives us the desired formula.

Number of non-negative integer sums

This problem is a direct application of the theorem.

You want to count the number of solution of the equation

$$x_1 + x_2 + \dots + x_k = n$$

with $x_i \geq 0$.

Again we can represent a solution using stars and bars. E.g. the solution $1 + 3 + 0 = 4$ for $n = 4$, $k = 3$ can be represented using $\star|\star\star\star|$.

It is easy to see, that this is exactly the stars and bars theorem. Therefore the solution is $\binom{n+k-1}{n}$.

Number of positive integer sums

A second theorem provides a nice interpretation for positive integers. Consider solutions to

$$x_1 + x_2 + \dots + x_k = n$$

with $x_i \geq 1$.

We can consider n stars, but this time we can put at most *one bar* between stars, since two bars between stars would represent $x_i = 0$, i.e. an empty box. There are $n-1$ gaps between stars to place $k-1$ bars, so the solution is $\binom{n-1}{k-1}$.

Number of lower-bound integer sums

This can easily be extended to integer sums with different lower bounds. I.e. we want to count the number of solutions for the equation

$$x_1 + x_2 + \cdots + x_k = n$$

with $x_i \geq a_i$.

After substituting $x'_i := x_i - a_i$ we receive the modified equation

$$(x'_1 + a_1) + (x'_2 + a_2) + \cdots + (x'_k + a_k) = n$$

$$\Leftrightarrow x'_1 + x'_2 + \cdots + x'_k = n - a_1 - a_2 - \cdots - a_k$$

with $x'_i \geq 0$. So we have reduced the problem to the simpler case with $x'_i \geq 0$ and again can apply the stars and bars theorem.

Number of upper-bound integer sums

With some help of the [Inclusion-Exclusion Principle](#), you can also restrict the integers with upper bounds. See the [Number of upper-bound integer sums](#) section in the corresponding article.

Practice Problems

- [Codeforces - Array](#)
- [Codeforces - Kyoya and Coloured Balls](#)
- [Codeforces - Colorful Bricks](#)
- [Codeforces - Two Arrays](#)
- [Codeforces - One-Dimensional Puzzle](#)

Contributors:

[Jakob Kogler](#) (70.73%) [jxu](#) (13.41%) [Mirco Paul](#) (6.1%) [Oleksandr Kulkov](#) (6.1%) [Mrunank Mistry](#) (3.66%)

Translated

From: e-maxx.ru

Generating all K -combinations

In this article we will discuss the problem of generating all K -combinations. Given the natural numbers N and K , and considering a set of numbers from 1 to N . The task is to derive all **subsets of size K** .

Generate next lexicographical K -combination

First we will generate them in lexicographical order. The algorithm for this is simple. The first combination will be $1, 2, \dots, K$. Now let's see how to find the combination that immediately follows this, lexicographically. To do so, we consider our current combination, and find the rightmost element that has not yet reached its highest possible value. Once finding this element, we increment it by 1, and assign the lowest valid value to all subsequent elements.

```
bool next_combination(vector<int>& a, int n) {
    int k = (int)a.size();
    for (int i = k - 1; i >= 0; i--) {
        if (a[i] < n - k + i + 1) {
            a[i]++;
            for (int j = i + 1; j < k; j++)
                a[j] = a[j - 1] + 1;
            return true;
        }
    }
    return false;
}
```

Generate all K -combinations such that adjacent combinations differ by one element

This time we want to generate all K -combinations in such an order, that adjacent combinations differ exactly by one element.

This can be solved using the [Gray Code](#): If we assign a bitmask to each subset, then by generating and iterating over these bitmasks with Gray codes, we can obtain our answer.

The task of generating K -combinations can also be solved using Gray Codes in a different way: Generate Gray Codes for the numbers from 0 to $2^N - 1$ and leave only those codes containing K 1s. The surprising fact is that in the resulting sequence of K set bits, any two neighboring masks (including the first and last mask - neighboring in a cyclic sense) - will differ exactly by two bits, which is our objective (remove a number, add a number).

Let us prove this:

For the proof, we recall the fact that the sequence $G(N)$ (representing the N^{th} Gray Code) can be obtained as follows:

$$G(N) = 0G(N - 1) \cup 1G(N - 1)^R$$

That is, consider the Gray Code sequence for $N - 1$, and prefix 0 before every term. And consider the reversed Gray Code sequence for $N - 1$ and prefix a 1 before every mask, and concatenate these two sequences.

Now we may produce our proof.

First, we prove that the first and last masks differ exactly in two bits. To do this, it is sufficient to note that the first mask of the sequence $G(N)$, will be of the form $N - K$ 0s, followed by K 1s. As the first bit is set as 0, after which $(N - K - 1)$ 0s follow, after which K set bits follow and the last mask will be of the form 1, then $(N - K)$ 0s, then $K - 1$ 1s. Applying the principle of mathematical induction, and using the formula for $G(N)$, concludes the proof.

Now our task is to show that any two adjacent codes also differ exactly in two bits, we can do this by considering our recursive equation for the generation of Gray Codes. Let us assume the content of the two halves formed by $G(N - 1)$ is true. Now we need to prove that the new consecutive pair formed at the junction (by the concatenation of these two halves) is also valid, i.e. they differ by exactly two bits.

This can be done, as we know the last mask of the first half and the first mask of the second half. The last mask of the first half would be 1, then $(N - K - 1)$ 0s, then $K - 1$ 1s. And the first mask of the second half would be 0, then $(N - K - 2)$ 0s would follow, and then $K - 1$ 1s. Thus, comparing the two masks, we find exactly two bits that differ.

The following is a naive implementation working by generating all 2^n possible subsets, and finding subsets of size K .

```
int gray_code (int n) {
    return n ^ (n >> 1);
}

int count_bits (int n) {
    int res = 0;
    for (; n; n >>= 1)
        res += n & 1;
    return res;
}

void all_combinations (int n, int k) {
    for (int i = 0; i < (1 << n); i++) {
        int cur = gray_code (i);
        if (count_bits(cur) == k) {
            for (int j = 0; j < n; j++) {
                if (cur & (1 << j))
                    cout << j + 1;
            }
            cout << "\n";
        }
    }
}
```

It's worth mentioning that a more efficient implementation exists that only resorts to building valid combinations and thus works in $O(N \cdot \binom{N}{K})$ however it is recursive in nature and for smaller values of N it probably has a larger constant than the previous solution.

The implementation is derived from the formula:

$$G(N, K) = 0G(N - 1, K) \cup 1G(N - 1, K - 1)^R$$

This formula is obtained by modifying the general equation to determine the Gray code, and works by selecting the subsequence from appropriate elements.

Its implementation is as follows:

```
vector<int> ans;

void gen(int n, int k, int idx, bool rev) {
    if (k > n || k < 0)
        return;

    if (!n) {
        for (int i = 0; i < idx; ++i) {
            if (ans[i])
                cout << i + 1;
        }
        cout << "\n";
        return;
    }

    ans[idx] = rev;
    gen(n - 1, k - rev, idx + 1, false);
    ans[idx] = !rev;
    gen(n - 1, k - !rev, idx + 1, true);
}

void all_combinations(int n, int k) {
    ans.resize(n);
}
```

```
    gen(n, k, 0, false);  
}
```

Contributors:

[Aditya Ramesh](#) (91.49%) [Jakob Kogler](#) (6.38%) [Oleksandr Kulkov](#) (2.13%)

Translated

From: e-maxx.ru

Placing Bishops on a Chessboard

Find the number of ways to place K bishops on an $N \times N$ chessboard so that no two bishops attack each other.

Algorithm

This problem can be solved using dynamic programming.

Let's enumerate the diagonals of the chessboard as follows: black diagonals have odd indices, white diagonals have even indices, and the diagonals are numbered in non-decreasing order of the number of squares in them. Here is an example for a 5×5 chessboard.

1	2	5	6	9
2	5	6	9	8
5	6	9	8	7
6	9	8	7	4
9	8	7	4	3

Let $D[i][j]$ denote the number of ways to place j bishops on diagonals with indices up to i which have the same color as diagonal i . Then $i = 1 \dots 2N-1$ and $j = 0 \dots K$.

We can calculate $D[i][j]$ using only values of $D[i-2]$ (we subtract 2 because we only consider diagonals of the same color as i). There are two ways to get $D[i][j]$. Either we place all j bishops on previous diagonals: then there are $D[i-2][j]$ ways to achieve this. Or we place one bishop on diagonal i and $j-1$ bishops on previous diagonals. The number of ways to do this equals the number of squares in diagonal i minus $j-1$, because each of $j-1$ bishops placed on previous diagonals will block one square on the current diagonal. The number of squares in diagonal i can be calculated as follows:

```
int squares (int i) {
    if (i & 1)
        return i / 4 * 2 + 1;
    else
        return (i - 1) / 4 * 2 + 2;
}
```

The base case is simple: $D[i][0] = 1$, $D[1][1] = 1$.

Once we have calculated all values of $D[i][j]$, the answer can be obtained as follows: consider all possible numbers of bishops placed on black diagonals $i=0 \dots K$, with corresponding numbers of bishops on white diagonals $K-i$. The bishops placed on black and white diagonals never attack each other, so the placements can be done independently. The index of the last black diagonal is $2N-1$, the last white one is $2N-2$. For each i we add $D[2N-1][i] * D[2N-2][K-i]$ to the answer.

Implementation

```
int bishop_placements(int N, int K)
{
    if (K > 2 * N - 1)
        return 0;

    vector<vector<int>> D(N * 2, vector<int>(K + 1));
    for (int i = 0; i < N * 2; ++i)
        D[i][0] = 1;
    D[1][1] = 1;
    for (int i = 2; i < N * 2; ++i)
```

```
        for (int j = 1; j <= K; ++j)
            D[i][j] = D[i-2][j] + D[i-2][j-1] * (squares(i) - j + 1);

    int ans = 0;
    for (int i = 0; i <= K; ++i)
        ans += D[N*2-1][i] * D[N*2-2][K-i];
    return ans;
}
```

Contributors:

[Mariia Mykhailova](#) (53.42%) [Jakob Kogler](#) (38.36%) [Oleksandr Kulkov](#) (8.22%)

Translated

From: e-maxx.ru

Balanced bracket sequences

A **balanced bracket sequence** is a string consisting of only brackets, such that this sequence, when inserted certain numbers and mathematical operations, gives a valid mathematical expression. Formally you can define balanced bracket sequence with:

- ϵ (the empty string) is a balanced bracket sequence.
- if s is a balanced bracket sequence, then so is (s) .
- if s and t are balanced bracket sequences, then so is st .

For instance $((()))$ is a balanced bracket sequence, but $()()$ is not.

Of course you can define other bracket sequences also with multiple bracket types in a similar fashion.

In this article we discuss some classic problems involving balanced bracket sequences (for simplicity we will only call them sequences): validation, number of sequences, finding the lexicographical next sequence, generating all sequences of a certain size, finding the index of sequence, and generating the k -th sequences. We will also discuss two variations for the problems, the simpler version when only one type of brackets is allowed, and the harder case when there are multiple types.

Balance validation

We want to check if a given string is balanced or not.

At first suppose there is only one type of bracket. For this case there exists a very simple algorithm. Let `depth` be the current number of open brackets. Initially `depth = 0`. We iterate over all character of the string, if the current bracket character is an opening bracket, then we increment `depth`, otherwise we decrement it. If at any time the variable `depth` gets negative, or at the end it is different from 0, then the string is not a balanced sequence. Otherwise it is.

If there are several bracket types involved, then the algorithm needs to be changed. Instead of a counter `depth` we create a stack, in which we will store all opening brackets that we meet. If the current bracket character is an opening one, we put it onto the stack. If it is a closing one, then we check if the stack is non-empty, and if the top element of the stack is of the same type as the current closing bracket. If both conditions are fulfilled, then we remove the opening bracket from the stack. If at any time one of the conditions is not fulfilled, or at the end the stack is not empty, then the string is not balanced. Otherwise it is.

Number of balanced sequences

Formula

The number of balanced bracket sequences with only one bracket type can be calculated using the [Catalan numbers](#). The number of balanced bracket sequences of length $2n$ (n pairs of brackets) is:

$$\frac{1}{n+1} \binom{2n}{n}$$

If we allow k types of brackets, then each pair can be of any of the k types (independently of the others), thus the number of balanced bracket sequences is:

$$\frac{1}{n+1} \binom{2n}{n} k^n$$

Dynamic programming

On the other hand these numbers can be computed using **dynamic programming**. Let $d[n]$ be the number of regular bracket sequences with n pairs of bracket. Note that in the first position there is always an opening bracket. And somewhere later is the corresponding closing bracket of the pair. It is clear that inside this pair there is a balanced bracket sequence, and similarly after this pair there is a balanced bracket sequence.

So to compute $d[n]$, we will look at how many balanced sequences of i pairs of brackets are inside this first bracket pair, and how many balanced sequences with $n - 1 - i$ pairs are after this pair. Consequently the formula has the form:

$$d[n] = \sum_{i=0}^{n-1} d[i] \cdot d[n-1-i]$$

The initial value for this recurrence is $d[0] = 1$.

Finding the lexicographical next balanced sequence

Here we only consider the case with one valid bracket type.

Given a balanced sequence, we have to find the next (in lexicographical order) balanced sequence.

It should be obvious, that we have to find the rightmost opening bracket, which we can replace by a closing bracket without violation the condition, that there are more closing brackets than opening brackets up to this position. After replacing this position, we can fill the remaining part of the string with the lexicographically minimal one: i.e. first with as much opening brackets as possible, and then fill up the remaining positions with closing brackets. In other words we try to leave as long as possible prefix unchanged, and the suffix gets replaced by the lexicographically minimal one.

To find this position, we can iterate over the character from right to left, and maintain the balance `depth` of open and closing brackets. When we meet an opening brackets, we will decrement `depth`, and when we meet a closing bracket, we increase it. If we are at some point meet an opening bracket, and the balance after processing this symbol is positive, then we have found the rightmost position that we can change. We change the symbol, compute the number of opening and closing brackets that we have to add to the right side, and arrange them in the lexicographically minimal way.

If we don't find a suitable position, then this sequence is already the maximal possible one, and there is no answer.

```
bool next_balanced_sequence(string & s) {
    int n = s.size();
    int depth = 0;
    for (int i = n - 1; i >= 0; i--) {
        if (s[i] == '(')
            depth--;
        else
            depth++;

        if (s[i] == '(' && depth > 0) {
            depth--;
            int open = (n - i - 1 - depth) / 2;
            int close = n - i - 1 - open;
            string next = s.substr(0, i) + ')' + string(open, '(') + string(close, ')');
            s.swap(next);
            return true;
        }
    }
    return false;
}
```

This function computes in $O(n)$ time the next balanced bracket sequence, and returns false if there is no next one.

Finding all balanced sequences

Sometimes it is required to find and output all balanced bracket sequences of a specific length n .

To generate then, we can start with the lexicographically smallest sequence $((\dots (()) \dots))$, and then continue to find the next lexicographically sequences with the algorithm described in the previous section.

However, if the length of the sequence is not very long (e.g. n smaller than 12), then we can also generate all permutations conveniently with the C++ STL function `next_permutation`, and check each one for balanceness.

Also they can be generated using the ideas we used for counting all sequences with dynamic programming. We will discuss the ideas in the next two sections.

Sequence index

Given a balanced bracket sequence with n pairs of brackets. We have to find its index in the lexicographically ordered list of all balanced sequences with n bracket pairs.

Let's define an auxiliary array $d[i][j]$, where i is the length of the bracket sequence (semi-balanced, each closing bracket has a corresponding opening bracket, but not every opening bracket has necessarily a corresponding closing one), and j is the current balance (difference between opening and closing brackets). $d[i][j]$ is the number of such sequences that fit the parameters. We will calculate these numbers with only one bracket type.

For the start value $i = 0$ the answer is obvious: $d[0][0] = 1$, and $d[0][j] = 0$ for $j > 0$. Now let $i > 0$, and we look at the last character in the sequence. If the last character was an opening bracket $($, then the state before was $(i - 1, j - 1)$, if it was a closing bracket $)$, then the previous state was $(i - 1, j + 1)$. Thus we obtain the recursion formula:

$$d[i][j] = d[i - 1][j - 1] + d[i - 1][j + 1]$$

$d[i][j] = 0$ holds obviously for negative j . Thus we can compute this array in $O(n^2)$.

Now let us generate the index for a given sequence.

First let there be only one type of brackets. We will use the counter `depth` which tells us how nested we currently are, and iterate over the characters of the sequence. If the current character $s[i]$ is equal to $($, then we increment `depth`. If the current character $s[i]$ is equal to $)$, then we must add $d[2n - i - 1][\text{depth} + 1]$ to the answer, taking all possible endings starting with a $($ into account (which are lexicographically smaller sequences), and then decrement `depth`.

Now let there be k different bracket types.

Thus, when we look at the current character $s[i]$ before recomputing `depth`, we have to go through all bracket types that are smaller than the current character, and try to place this bracket into the current position (obtaining a new balance $\text{ndepth} = \text{depth} \pm 1$), and add the number of ways to finish the sequence (length $2n - i - 1$, balance ndepth) to the answer:

$$d[2n - i - 1][\text{ndepth}] \cdot k^{\frac{2n - i - 1 - \text{ndepth}}{2}}$$

This formula can be derived as follows: First we "forget" that there are multiple bracket types, and just take the answer $d[2n - i - 1][\text{ndepth}]$. Now we consider how the answer will change if we have k types of brackets. We have $2n - i - 1$ undefined positions, of which ndepth are already predetermined because of the opening brackets. But all the other brackets $((2n - i - 1 - \text{ndepth}) / 2$ pairs) can be of any type, therefore we multiply the number by such a power of k .

Finding the k -th sequence

Let n be the number of bracket pairs in the sequence. We have to find the k -th balanced sequence in lexicographically sorted list of all balanced sequences for a given k .

As in the previous section we compute the auxiliary array $d[i][j]$, the number of semi-balanced bracket sequences of length i with balance j .

First, we start with only one bracket type.

We will iterate over the characters in the string we want to generate. As in the previous problem we store a counter `depth`, the current nesting depth. At each position, we have to decide whether to place an opening or a closing bracket. To place an opening bracket, $d[2n - i - 1][\text{depth} + 1] \geq k$ must be true. If so, we increment the counter `depth`, and move on to the next character. Otherwise, we decrement k by $d[2n - i - 1][\text{depth} + 1]$, place a closing bracket, and move on.

```
string kth_balanced(int n, int k) {
    vector<vector<int>> d(2*n+1, vector<int>(n+1, 0));
    d[0][0] = 1;
```

```

for (int i = 1; i <= 2*n; i++) {
    d[i][0] = d[i-1][1];
    for (int j = 1; j < n; j++)
        d[i][j] = d[i-1][j-1] + d[i-1][j+1];
    d[i][n] = d[i-1][n-1];
}

string ans;
int depth = 0;
for (int i = 0; i < 2*n; i++) {
    if (depth + 1 <= n && d[2*n-i-1][depth+1] >= k) {
        ans += '(';
        depth++;
    } else {
        ans += ')';
        if (depth + 1 <= n)
            k -= d[2*n-i-1][depth+1];
        depth--;
    }
}
return ans;
}

```

Now let there be k types of brackets. The solution will only differ slightly in that we have to multiply the value $d[2n - i - 1][ndepth]$ by $k^{(2n-i-1-ndepth)/2}$ and take into account that there can be different bracket types for the next character.

Here is an implementation using two types of brackets: round and square:

```

string kth_balanced2(int n, int k) {
    vector<vector<int>> d(2*n+1, vector<int>(n+1, 0));
    d[0][0] = 1;
    for (int i = 1; i <= 2*n; i++) {
        d[i][0] = d[i-1][1];
        for (int j = 1; j < n; j++)
            d[i][j] = d[i-1][j-1] + d[i-1][j+1];
        d[i][n] = d[i-1][n-1];
    }

    string ans;
    int shift, depth = 0;

    stack<char> st;
    for (int i = 0; i < 2*n; i++) {
        // '('
        shift = ((2*n-i-1-depth-1) / 2);
        if (shift >= 0 && depth + 1 <= n) {
            int cnt = d[2*n-i-1][depth+1] << shift;
            if (cnt >= k) {
                ans += '(';
                st.push('(');
                depth++;
                continue;
            }
            k -= cnt;
        }

        // ')'
        shift = ((2*n-i-1-depth+1) / 2);
        if (shift >= 0 && depth && st.top() == '(') {
            int cnt = d[2*n-i-1][depth-1] << shift;
            if (cnt >= k) {
                ans += ')';
                st.pop();
                depth--;
                continue;
            }
            k -= cnt;
        }

        // '['
        shift = ((2*n-i-1-depth-1) / 2);
        if (shift >= 0 && depth + 1 <= n) {
            int cnt = d[2*n-i-1][depth+1] << shift;
            if (cnt >= k) {
                ans += '[';
                st.push('[');
                depth++;
            }
        }
    }
}

```

```
        continue;
    }
    k -= cnt;
}

// ']'
ans += ']' ;
st.pop();
depth--;
}
return ans;
}
```

Contributors:

[Jakob Kogler](#) (88.6%) [Sabit Zahin](#) (4.41%) [Ahmet Ala](#) (3.68%) [Oleksandr Kulkov](#) (2.21%) [Michael Clock](#) (0.37%) [gnud-gnaoh](#) (0.37%) [wiktorkuchta](#) (0.37%)

Translated From: e-maxx.ru

Counting labeled graphs

Labeled graphs

Let the number of vertices in a graph be n . We have to compute the number G_n of labeled graphs with n vertices (labeled means that the vertices are marked with the numbers from 1 to n). The edges of the graphs are considered undirected, and loops and multiple edges are forbidden.

We consider the set of all possible edges of the graph. For each edge (i, j) we can assume that $i < j$ (because the graph is undirected, and there are no loops). Therefore the set of all edges has the cardinality $\binom{n}{2}$, i.e. $\frac{n(n-1)}{2}$.

Since any labeled graph is uniquely determined by its edges, the number of labeled graphs with n vertices is equal to:

$$G_n = 2^{\frac{n(n-1)}{2}}$$

Connected labeled graphs

Here, we additionally impose the restriction that the graph has to be connected.

Let's denote the required number of connected graphs with n vertices as C_n .

We will first discuss how many **disconnected** graphs exists. Then the number of connected graphs will be G_n minus the number of disconnected graphs. Even more, we will count the number of **disconnected, rooted graphs**. A rooted graph is a graph, where we emphasize one vertex by labeling it as root. Obviously we have n possibilities to root a graph with n labeled vertices, therefore we will need to divide the number of disconnected rooted graphs by n at the end to get the number of disconnected graphs.

The root vertex will appear in a connected component of size $1, \dots, n-1$. There are $k \binom{n}{k} C_k G_{n-k}$ graphs such that the root vertex is in a connected component with k vertices (there are $\binom{n}{k}$ ways to choose k vertices for the component, these are connected in one of C_k ways, the root vertex can be any of the k vertices, and the remainder $n-k$ vertices can be connected/disconnected in any way, which gives a factor of G_{n-k}). Therefore the number of disconnected graphs with n vertices is:

$$\frac{1}{n} \sum_{k=1}^{n-1} k \binom{n}{k} C_k G_{n-k}$$

And finally the number of connected graphs is:

$$C_n = G_n - \frac{1}{n} \sum_{k=1}^{n-1} k \binom{n}{k} C_k G_{n-k}$$

Labeled graphs with k connected components

Based on the formula from the previous section, we will learn how to count the number of labeled graphs with n vertices and k connected components.

This number can be computed using dynamic programming. We will compute $D[i][j]$ - the number of labeled graphs with i vertices and j components - for each $i \leq n$ and $j \leq k$.

Let's discuss how to compute the next element $D[n][k]$ if we already know the previous values. We use a common approach, we take the last vertex (index n). This vertex belongs to some component. If the size of this component be s , then there are $\binom{n-1}{s-1}$ ways to choose such a set of vertices, and C_s ways to connect them. After removing this component from the graph we have $n - s$ remaining vertices with $k - 1$ connected components. Therefore we obtain the following recurrence relation:

$$D[n][k] = \sum_{s=1}^n \binom{n-1}{s-1} C_s D[n-s][k-1]$$

Contributors:

[Jakob Kogler](#) (89.47%) [Oleksandr Kulkov](#) (10.53%)